Algorithmic Architecture for Humanized MIDI Comping in Python

Introduction to Algorithmic MIDI Humanization

The translation of symbolic musical information into expressive, lifelike performance data represents a significant frontier in algorithmic music composition.[1] While computational systems excel at generating accurate harmonic structures and parsing complex chord symbols from JSON configurations (such as Fm9 or Dbmaj7), the resulting digital output frequently suffers from mechanical rigidity.[3] When utilizing foundational libraries such as music21 to calculate spread voicings and midiutil to write the corresponding MIDI events, the default methodology generates static block chords. These chords are characterized by simultaneous note onsets, mathematically perfect durations, and uniform velocity values.[5] In genres that rely heavily on micro-timing, dynamic contouring, and syncopated grooves—most notably R&B, Neo-soul, and modern Pop—this robotic precision shatters the aesthetic illusion of a human instrumentalist.[3]

To bridge the chasm between static harmonic arrays and rhythmic, stylized performances, an intermediate "comping" (accompanying) engine must be architected. The primary objective of this engine is to serve as an algorithmic humanization layer, intercepting static blocks of MIDI pitches and translating them into dynamic, temporally nuanced rhythmic expressions. This architectural endeavor operates under strict technical constraints: it must rely entirely on pure Python data structures and mathematical operations, utilizing only the midiutil library for the final output generation. The midiutil framework interfaces with MIDI data via the addNote(track, channel, pitch, time, duration, velocity) method.[6] Because this method utilizes floating-point values for the time and duration parameters to represent metrical beats, sub-beat precision can be mathematically manipulated to induce complex rhythmic phenomena such as MPC-style swing, piano chord rolling, and humanized temporal drift.[8] Furthermore, velocity values, which are constrained to an integer range of 0 to 127, can be algorithmically randomized using advanced stochastic models to simulate the kinetic energy and biomechanical inconsistencies of physical piano playing.[9]

This exhaustive research report presents a comprehensive architectural blueprint for a Python-based MIDI comping engine. It details the structural schema for rhythm pattern data objects, the precise mathematical models required for swing and micro-timing (including the emulation of the famous J Dilla drunken feel), the stochastic algorithms governing humanized velocity curves, and the formulaic logic necessary to achieve millisecond-accurate staggered chord onsets.[10]

Stylistic Foundations of Neo-Soul and R&B Comping

Before engineering the data structures responsible for generating rhythm, it is imperative to

understand the musical and biomechanical principles that define Neo-soul and R&B keyboard performances. The unique aesthetic of Neo-soul comping relies on a combination of advanced harmonic extensions (major 7ths, 9ths, 11ths, and diminished passing chords) and highly specific rhythmic phrasing.[3] When a human pianist accompanies a vocalist or a drum groove in these genres, they rarely sustain a single block chord for an entire measure. Instead, the harmonic information is distributed across the metrical grid through syncopation, rhythmic anticipation, and the dynamic interplay between the left and right hands.[14]

The left hand typically provides the harmonic foundation by playing the root note or a bass shell voicing, serving as an anchor on strong metrical subdivisions.[3] The right hand, meanwhile, handles the upper harmonic extensions and engages in complex rhythmic dialogue. A defining characteristic of this style is the "push," a rhythmic technique where the upper chord is struck a sixteenth or eighth note before the downbeat, effectively anticipating the harmony of the upcoming measure.[7] Additionally, human performances are peppered with ghost notes—extremely quiet, percussive strikes that serve to propel the groove forward without muddying the harmonic space.[16]

To algorithmically replicate these nuances, the comping engine must be capable of deconstructing a static array of pitches into discrete, stylistically appropriate rhythmic events. The architecture must separate the bass register from the upper voicings and apply unique temporal and dynamic rules to each.[1] Furthermore, the engine must account for the natural velocity hierarchies inherent in human playing, where downbeats naturally receive more kinetic energy than off-beats, and where the outer fingers of the hand strike the keys with more force than the inner fingers.[19]

Architecting the Metrical Data Structure

The foundation of the comping engine is a robust, scalable data structure designed to capture the intricacies of a rhythm pattern while remaining abstract enough to map to any incoming chord progression. Rather than hard-coding rhythms into procedural generation loops, the optimal approach involves defining discrete musical events as objects.[4] The architecture separates the continuous temporal space of a measure into a sequence of "Hits." Each hit specifies the exact sub-beat where a note should be struck, the metrical duration it should be held, its relative kinetic energy (defined as a velocity multiplier), and the specific subset of the chord's pitches it should trigger.[1]

Given the constraint to utilize pure Python, the architecture leverages the dataclasses module to ensure type safety, memory efficiency, and maximum code readability. This object-oriented approach allows developers to build a comprehensive library of groove templates that can be dynamically applied to the output of the JSON chord parser.

The CompingHit and CompingPattern Schemas

The primary data unit is the CompingHit class. This object contains the floating-point instructions necessary to interface with midiutil's sub-beat resolution capabilities.[8]

Parameter	**Data Type**	**Algorithmic Function**
beat_offset	float	Defines the exact floating-point position within the measure where the hit occurs. A value of 0.0 represents the absolute downbeat (beat 1), while 1.5 represents the exact eighth-note subdivision separating beat 2 and beat 3.
duration	float	Dictates the sustain length of the hit, expressed in metrical beats rather than absolute seconds, ensuring temporal consistency regardless of global tempo changes.
velocity_mult	float	A scalar multiplier (typically ranging from 0.1 to 1.5) applied to the system's baseline velocity. This parameter distinguishes thunderous accented downbeats from whispering, syncopated ghost notes.
voice_group	str	A string identifier designating which subset of the parsed chord should be triggered. Expected values include "bass" for the lowest root note, "chord"

		for the upper extensions, or "all" for full-handed block strikes.

The aggregate of these hits forms a CompingPattern, which represents a complete metrical cycle—typically one or two measures in length.[21] The implementation of these schemas in pure Python is detailed in the following code block.

Python

```python
from dataclasses import dataclass
from typing import List, Dict, Tuple

@dataclass
class CompingHit:
    """
    Represents a single rhythmic strike within a defined comping pattern.
    Provides the necessary floating-point parameters for sub-beat precision.
    """
    beat_offset: float
    duration: float
    velocity_mult: float
    voice_group: str  # Options: 'bass', 'chord', 'all'

@dataclass
class CompingPattern:
    """
    Represents a complete metrical groove template.
    Encapsulates a sequence of CompingHits over a defined beat length.
    """
    name: str
    length_in_beats: float
    hits: List[CompingHit]
```

Encoding Stylistic Grooves

By utilizing this abstraction, complex R&B and Neo-soul rhythms can be codified into reusable templates. For example, a classic Neo-soul syncopated bounce relies heavily on the

independence of the bass note from the upper chord.[14] The pattern typically begins with a heavily accented bass note anchoring the downbeat. This is followed by a staccato chord strike on beat two, a quiet ghosted anticipation on the upbeat of beat two (beat 2.5), and a highly emphasized, syncopated chord on the upbeat of beat three (beat 3.5) that sustains through the end of the measure, creating the characteristic "push" effect.[7]

Conversely, an R&B ballad pattern requires a much more expansive, legato approach. In this scenario, the bass note sustains for the entirety of the measure, while the upper chords ring out in slow, overlapping cascades.[24] The flexibility of the CompingHit schema allows both of these disparate styles to be defined natively.

Python

```python
# Definition of a Neo-Soul syncopated bounce pattern featuring the characteristic "push"
neo_soul_push = CompingPattern(
    name="Neo_Soul_Push",
    length_in_beats=4.0,
    hits=
)

# Definition of a classic R&B Ballad arpeggiation prioritizing sustained legato
rnb_ballad = CompingPattern(
    name="RnB_Ballad",
    length_in_beats=4.0,
    hits=
)
```

By abstracting rhythm into these scalable data structures, the comping engine can effortlessly ingest any list of MIDI pitches generated by music21. The engine separates the lowest integer pitch, assigns it to the "bass" voice group, maps the remaining pitches to the "chord" voice group, and iterates the target CompingPattern across the song's harmonic timeline.[26]

The Mathematics of Micro-Timing and Swing

In standard digital music production environments, mechanical timing is dictated by a strict, linear quantization grid. Every note occurs at a mathematically perfect subdivision of the measure.[5] However, human grooves—particularly those rooted in blues, jazz, and hip-hop—rely on swing. Swing is the deliberate, systematic temporal displacement of specific notes within the metrical grid, creating a rolling, cyclical rhythm that standard quantization cannot emulate.[28]

In software systems designed to mimic human groove, such as the legendary Akai MPC series engineered by Roger Linn, swing is not achieved through random timing errors. Instead, it is applied through a precise mathematical formula that delays the even-numbered subdivisions (the off-beats or "ands") while ensuring the odd-numbered subdivisions (the strong downbeats) remain strictly anchored to the quantization grid.[30] This interplay between anchored beats and delayed off-beats is the fundamental engine of groove.

The Geometry and Ratios of Swing

Swing is universally measured as a percentage representing the ratio of time allotted to the first subdivision versus the second subdivision within a single beat.[29] Understanding this ratio is critical to engineering the mathematical transformation of floating-point time values in midiutil.

Swing Percentage	Mathematical Ratio	Musical Characteristics
50% (Straight)	1:1 Ratio	The downbeat and upbeat divide the metrical interval equally. No delay is applied. Standard for strict pop, rock, and classical sequencing.[30]
54% - 62% (Dilla/Drunk)	Variable Ratio	The hallmark of J Dilla's micro-timing and modern Neo-soul. The off-beats are delayed slightly, sitting uncomfortably but beautifully between straight eighths and triplet eighths. This is often referred to as septuplet or quintuplet swing.[11]
66.6% (Perfect Triplet)	2:1 Ratio	The downbeat consumes exactly two-thirds of the beat interval, and the upbeat consumes one-third. The upbeat is delayed until the third partial of an underlying triplet grid.[30]

75% (Hard Shuffle)	3:1 Ratio	The downbeat consumes three-quarters of the interval. The upbeat is pushed to the final sixteenth note partial, creating a sharp, rigid dotted rhythm.[28]

To apply this computationally within the parameters of midiutil, where the time parameter is represented strictly as a float representing metrical beats, an algorithmic transformation must be applied to the beat_offset of every instantiated CompingHit.[8]

Formulating the Swing Algorithm

Let t represent the original, strictly quantized beat offset of the musical event. Let Q represent the swing quantization grid in beats. For an eighth-note swing feel, $Q = 0.5$. For a sixteenth-note swing feel, which is standard for most Neo-soul and hip-hop grooves, $Q = 0.25$.[35] Let S represent the desired swing percentage, expressed as a floating-point value between 50.0 and 100.0.

The algorithm must first determine the positional step index k of the note on the selected quantization grid:

$$k = \lfloor \frac{t}{Q} \rfloor$$

If k evaluates to an even integer, the note falls on a strong grid line (e.g., beat 1.0, beat 1.5, beat 2.0). According to the principles established by Roger Linn, these notes serve as the metrical anchor and must **not** be swung.[30] If k evaluates to an odd integer, the note represents an off-beat (e.g., beat 1.25, beat 1.75) and must be mathematically delayed.

The maximum possible delay for an off-beat—which would push it entirely onto the subsequent grid point—is exactly equal to Q. However, the algorithm only pushes the note halfway toward the next grid point, meaning the maximum theoretical push is $Q/2$. The percentage of this maximum delay that is actually applied is calculated based on how far the swing percentage S exceeds the standard 50% baseline:

$$\text{Delay Factor} = \frac{S - 50}{50}$$

Therefore, the comprehensive mathematical formula for calculating the newly delayed, humanized time t' for any odd k is:

$$t' = t + \left(\frac{S - 50}{50}\right) \times \left(\frac{Q}{2}\right)$$

Note on formula efficacy: If a swing value of 66.66% is supplied, the delay factor evaluates to approximately 0.333. Multiplied by the half-grid $Q/2$, it shifts the sixteenth note precisely onto the third partial of the triplet grid, resulting in mathematically perfect triplet swing.

Pure Python Implementation of Sub-Beat Swing

The following algorithmic implementation translates this mathematical logic into pure Python float manipulation. It respects the sub-beat precision constraints of midiutil and introduces a microscopic epsilon factor to prevent catastrophic floating-point inaccuracies during the floor division process.[8]

Python

```python
import math

def apply_swing(time_in_beats: float, swing_percent: float = 60.0, swing_grid: float = 0.25) -> float:
    """
    Applies an MPC-style mathematical swing delay to a specific float time value.
    This function modifies the micro-timing of off-beats while preserving downbeats.

    :param time_in_beats: The original, rigidly quantized float time in beats (e.g., 1.25).
    :param swing_percent: The swing ratio, ranging from 50.0 (straight) to 75.0 (hard shuffle).
    :param swing_grid: The grid resolution. 0.5 designates 8th notes; 0.25 designates 16th notes.
    :return: The mathematically delayed, micro-timed float offset ready for midiutil.
    """
    # Swing percentages of 50 or below represent straight time; bypass logic
    if swing_percent <= 50.0:
        return time_in_beats
```

```
    # Clamp the swing percentage to a reasonable aesthetic maximum
    # Values above 80% usually push the note past the subsequent metrical partial
    swing_percent = min(max(swing_percent, 50.0), 80.0)

    # Calculate the integer step number on the quantization grid
    # An epsilon is added to prevent floats like 0.24999999 from flooring incorrectly to 0
    epsilon = 1e-6
    step = math.floor((time_in_beats + epsilon) / swing_grid)

    # Determine if the step is an off-beat (odd number)
    if step % 2!= 0:
        # Calculate the delay ratio (0.0 to 1.0 scale relative to the half-grid)
        delay_ratio = (swing_percent - 50.0) / 50.0

        # The absolute maximum we can push the note is exactly halfway to the next grid point
        max_push = swing_grid / 2.0

        # Calculate the final micro-timing offset
        swing_offset = delay_ratio * max_push

        # Return the delayed float time
        return time_in_beats + swing_offset

    # If the step is even (downbeat), return the original unaltered time
    return time_in_beats
```

This logic guarantees that downbeats retain their anchor positions, preserving the core tempo of the track, while the off-beats are nudged backward. This precise manipulation yields the characteristic "lopsided" groove inherent to J Dilla-inspired hip-hop and Neo-soul.[11]

Algorithmic Humanization of Velocity and Dynamics

A critical flaw in standard algorithmic composition is the reliance on static or uniformly randomized MIDI velocity. Pushing a flat velocity value of 100 uniformly across a complex spread chord lacks the dynamic contour produced by the anatomy of a human hand striking physical piano keys.[5] Real instrumentalists utilize a highly structured hierarchy of accents based primarily on metrical positioning.[19] Downbeats receive the most kinetic energy, backbeats (beats 2 and 4 in standard time) are accented but structurally differentiated, and off-beats are often reduced to barely audible "ghost notes".[16]

Furthermore, when a human attempts to play a note at a specific dynamic level, the resulting velocity is never mathematically perfect. However, it does not conform to a uniform random

distribution (e.g., simply adding or subtracting a random integer between 1 and 20). Instead, human biomechanical error conforms to a normal (Gaussian) distribution, clustering closely around the intended target with infrequent outlier errors.[10]

The Metrical Velocity Accent Matrix

To replicate this physical reality, the comping engine must first establish a base velocity depending entirely on where the CompingHit falls within the measure. In a standard 4/4 time signature, the beats carry the following hierarchical weight, which must be algorithmically assigned before stochastic randomization occurs [20]:

Metrical Position	**Musical Function**	**Base Target Velocity (Scale: 0-127)**
Beat 1.0	Primary Downbeat (Maximum Energy)	105 - 110
Beat 3.0	Secondary Downbeat (Strong Anchor)	95 - 100
Beats 2.0 & 4.0	Backbeats (Medium Snare Placement)	85 - 90
8th Note Off-beats	Weak Syncopations (e.g., 1.5, 2.5)	70 - 75
16th Note Off-beats	Ghost Notes (Minimal Energy)	40 - 50

Biomechanical Constraints and Gaussian Distribution

Beyond metrical positioning, the engine must account for the anatomy of the human hand. When a pianist strikes a spread chord, the outer fingers (the thumb and pinky) naturally fall with

more weight than the inner fingers.[16] Therefore, the algorithm must identify the lowest pitch (bass) and the highest pitch (melody) within the chord array and apply a positive scalar to their velocity, while slightly suppressing the inner harmonic voices.

Once the final target velocity is calculated via metrical position and biomechanical weighting, the algorithm utilizes Python's random.gauss(mu, sigma) function to apply human error.[12] In this formula, μ (mu) represents the calculated target velocity, and σ (sigma) represents the standard deviation—essentially the player's consistency level.[10] For a highly trained studio pianist, σ might be set to a tight value of 3.0. For a looser, more organic Neo-soul feel, σ might be expanded to 7.0 or 8.0, allowing for wider dynamic variance.

Pure Python Implementation of Velocity Humanization

The following algorithmic sequence isolates the metrical grid position, calculates the physiological hand weights, and outputs a Gaussian-distributed velocity integer clamped to the strict 1-127 MIDI standard.[9]

Python

```python
import random

def get_metrical_base_velocity(beat_offset: float, base_vel: int = 90) -> float:
    """
    Determines the baseline velocity target based on the exact metrical grid position.
    Simulates the natural accent hierarchy of a human instrumentalist in 4/4 time.
    """
    # Normalize the beat offset to a strict 0.0 - 3.99 range for a 4/4 measure
    beat_in_measure = beat_offset % 4.0

    # Define a helper function to safely compare floats representing metrical positions
    def is_close(a: float, b: float, tol: float = 0.05) -> bool:
        return abs(a - b) < tol

    # Evaluate the metrical hierarchy
    if is_close(beat_in_measure, 0.0):
        return base_vel * 1.15      # Primary downbeat receives a 15% boost
    elif is_close(beat_in_measure, 2.0):
        return base_vel * 1.05      # Secondary downbeat receives a 5% boost
    elif is_close(beat_in_measure, 1.0) or is_close(beat_in_measure, 3.0):
```

```
        return base_vel * 0.95      # Backbeats are slightly pulled back
    elif (beat_in_measure % 0.5) < 0.05:
        return base_vel * 0.80      # Standard 8th note off-beats
    else:
        return base_vel * 0.55      # 16th subdivisions function as quiet ghost notes

def humanize_velocity(
    beat_offset: float,
    pattern_multiplier: float,
    voice_index: int,
    total_voices: int,
    base_velocity: int = 90,
    humanize_amount: float = 5.0
) -> int:
    """
    Calculates a stochastically humanized integer velocity for a specific note in a chord.

    :param beat_offset: Metrical float position of the musical hit.
    :param pattern_multiplier: Dynamic multiplier dictated by the CompingHit schema.
    :param voice_index: Integer index of the pitch within the chord (0 = bass, highest = melody).
    :param total_voices: Total number of discrete pitches in the chord array.
    :param base_velocity: Base dynamic level desired for the track.
    :param humanize_amount: The standard deviation (sigma) for Gaussian randomization.
    :return: A MIDI-compliant integer between 1 and 127.
    """
    # 1. Establish initial target via metrical weighting
    target = get_metrical_base_velocity(beat_offset, base_velocity)

    # 2. Apply the dynamic multiplier explicitly defined by the pattern's composer
    target *= pattern_multiplier

    # 3. Apply Biomechanical Hand Weighting
    # Human hands naturally emphasize the outer voices of a chord block
    if total_voices > 1:
        if voice_index == 0:
            target *= 1.12  # Thumb/Pinky on bass gets a 12% boost
        elif voice_index == total_voices - 1:
            target *= 1.15  # Pinky on melody gets a 15% boost to cut through the mix
        else:
            target *= 0.88  # Inner structural voices are suppressed by 12%

    # 4. Apply Stochastic Gaussian Randomization
    # random.gauss accurately simulates the bell curve of human performance error
    final_velocity = random.gauss(mu=target, sigma=humanize_amount)
```

```
    # 5. Clamp the result to valid MIDI specifications
    # midiutil will fail or behave unpredictably if passed floats or out-of-bounds integers
    return max(1, min(127, int(round(final_velocity))))
```

This multi-tiered mathematical approach ensures the velocity dynamics breathe cyclically with the measure, mirror the biomechanical emphasis of the human hand, and feature a mathematically sound approximation of performance inconsistency.[9]

Temporal Displacement: Chord Rolling and Staggered Onsets

When a human pianist executes a spread voicing—for example, a dense 5-note minor 9th chord spanning three octaves—the physical keys are not depressed at the exact same millisecond.[27] The physical mechanics of the wrist and fingers render absolute simultaneity impossible. Depending on the aesthetic requirements of the genre, a pianist will intentionally "roll" the chord, usually striking the lowest note first and rapidly sweeping upward through the higher keys.[14] In R&B, Gospel, and Neo-soul piano, this upward roll creates a lush, cascading harmonic effect that static MIDI block chords inherently lack.[3]

To algorithmically simulate a pianist's hand rolling a chord, the time parameter passed to midiutil.addNote for each note within the same chord array must be staggered sequentially by 5 to 20 milliseconds from bottom to top.[27]

The Millisecond-to-Beat Conversion Mathematics

Because the project constraint mandates the exclusive use of midiutil, absolute temporal values (such as seconds or milliseconds) are completely abstracted. The midiutil.addNote method requires the time argument to be formatted strictly in *beats* relative to the track's designated tempo.[6] Therefore, staggering a note by 15 milliseconds requires the algorithm to dynamically calculate what fraction of a metrical beat 15 milliseconds represents at the current BPM.

The duration of a single beat (a standard quarter note) expressed in milliseconds is defined by the following division:

$$\text{ms per beat} = \frac{60,000}{\text{BPM}}$$

38

To isolate the float value representing a specific millisecond offset within a beat, the relationship is inverted:

$$\text{Beat Offset} = \frac{\text{Target ms}}{\text{ms per beat}}$$

Simplified into a single linear equation, the conversion from milliseconds to a beat offset float is:

$$\text{Beat Offset} = \text{Target ms} \times \frac{\text{BPM}}{60,000}$$

Table: Millisecond to Beat Float Conversions Across Tempos

Track BPM	Duration of 1 Beat (ms)	10ms Stagger in Beats	20ms Stagger in Beats
70 BPM (Slow R&B)	857.14 ms	0.0116 beats	0.0233 beats
90 BPM (Neo-Soul)	666.66 ms	0.0150 beats	0.0300 beats
120 BPM (Pop/Dance)	500.00 ms	0.0200 beats	0.0400 beats

As the track's tempo increases, a static 10ms physical delay naturally consumes a much larger fraction of the metrical beat. By calculating this dynamically based on the current BPM parameter, the temporal humanization remains physically consistent regardless of how the producer alters the overarching track tempo.[39]

Pure Python Implementation of Chord Rolling

The algorithmic approach for chord rolling first requires sorting the chord pitches numerically from lowest to highest. It then establishes the baseline float time for the bass note (which anchors the chord rhythmically) and iteratively accumulates the calculated beat offset for each subsequent upper voice.[27] To prevent the roll from sounding artificially perfect, Gaussian jitter is added to the millisecond gap between each strike.

Python

```python
def calculate_roll_offsets(
    pitches: List[int],
    base_time: float,
    bpm: float = 90.0,
    roll_ms_per_note: float = 12.0,
    randomize_roll: bool = True
) -> List[float]:
    """
    Calculates progressively staggered float time values to simulate a pianist rolling a chord.

    :param pitches: List of MIDI pitch integers, representing the chord.
    :param base_time: The starting float time in beats for the lowest note (post-swing).
    :param bpm: The global tempo of the track, necessary for millisecond conversion.
    :param roll_ms_per_note: Average target milliseconds to delay each ascending note.
    :param randomize_roll: If True, injects subtle Gaussian jitter into the roll gap.
    :return: A list of float times corresponding precisely to each pitch index.
    """
    # Sort pitches ascending to guarantee a bottom-to-top harmonic roll
    sorted_pitches = sorted(pitches)
    time_offsets =

    # Calculate the foundational beat equivalent of the targeted millisecond gap
    base_beat_gap = roll_ms_per_note * (bpm / 60000.0)

    current_time = base_time
    for i in range(len(sorted_pitches)):
        if i == 0:
            # The root/bass note strictly anchors the chord exactly on the intended time
            time_offsets.append(current_time)
        else:
            # Determine the temporal gap to the next ascending note
            if randomize_roll:
                # Add human jitter to the roll speed using a tight standard deviation of 2.5ms
                jitter_ms = random.gauss(mu=roll_ms_per_note, sigma=2.5)
                # Ensure severe jitter doesn't reverse the roll (clamp to a 2ms minimum)
                jitter_ms = max(2.0, jitter_ms)

                # Convert the jittered millisecond value to a float beat offset
                beat_gap = jitter_ms * (bpm / 60000.0)
            else:
                beat_gap = base_beat_gap
```

```
        # Accumulate the offset and append to the list
        current_time += beat_gap
        time_offsets.append(current_time)

    return time_offsets
```

This iterative accumulation accurately mimics the physiological behavior of the hand rolling consecutively across a keyboard. The addition of Gaussian jitter to the micro-timing gap replicates the subtle inconsistencies of muscle twitch dynamics, ensuring no two chords roll at exactly the same speed.[5]

Synthesis: The Complete Comping Engine

The disparate elements of the architecture—the metrical data schema, the MPC swing mathematics, the stochastic velocity generation, and the BPM-dependent temporal rolling—must now be unified into a cohesive, highly efficient comping engine.

The engine is engineered to intercept a list of static MIDI note arrays immediately after they are generated by the music21 parser. Instead of pushing these blocks directly to the midiutil.addNote function, the engine translates the pitch block across the selected CompingPattern structure, recursively resolving the micro-timing, displacement, and dynamics for every single note.

Operational Workflow of the Engine

1. **Voice Separation:** In modern Neo-soul production, the left hand (bass) and right hand (chords) function almost autonomously.[3] The engine sorts the array, excises the lowest pitch integer to form the bass voice, and leaves the remaining integers as the upper harmonic cluster.
2. **Pattern Iteration:** The function iterates through every CompingHit defined within the selected pattern template.
3. **Algorithmic Resolution:** For every hit, the global measure time is calculated. The MPC swing algorithm evaluates the position and applies delay if necessary. The chord roll function calculates the individual stagger for every pitch triggered by the hit. Finally, the velocity engine evaluates the metrical weight, applies Gaussian randomization, and clamps the output.[6]
4. **MIDI Translation:** All algorithmic permutations are strictly resolved into primitive Python floats and integers before invoking addNote, guaranteeing midiutil receives highly precise, pre-humanized data without requiring structural modifications to the library itself.[6]

Pure Python Implementation of the Master Rendering

Loop

Python

```python
from typing import List
# Note: Assumes CompingHit, CompingPattern, apply_swing, humanize_velocity,
# and calculate_roll_offsets are active in the namespace.

def render_comping_block(
    midi_object,              # Instantiated object from midiutil.MIDIFile
    track: int,
    channel: int,
    start_measure_time: float,  # Global absolute start time of the measure in beats
    bpm: float,
    chord_pitches: List[int],
    pattern: CompingPattern,
    swing_percent: float = 62.0,
    base_velocity: int = 85
) -> None:
    """
    Translates a static block of parsed pitches into a dynamic, rhythmic,
    and humanized performance, writing events directly into the midiutil object.

    :param midi_object: The active MIDIFile instance.
    :param track: MIDI track designation (0-15).
    :param channel: MIDI channel designation (0-15).
    :param start_measure_time: The global beat where the current measure begins.
    :param bpm: The track tempo, required for millisecond conversions.
    :param chord_pitches: The raw, unformatted list of MIDI pitch integers from JSON.
    :param pattern: The selected CompingPattern template object.
    :param swing_percent: The desired MPC swing ratio (50.0 to 80.0).
    :param base_velocity: The baseline dynamic intensity of the performance.
    """
    if not chord_pitches:
        return

    # 1. Voice Separation Logic
    chord_pitches = sorted(chord_pitches)
    bass_pitch = chord_pitches

    # Isolate upper extensions; if the input is a single note, default it to the bass
    upper_voices = chord_pitches[1:] if len(chord_pitches) > 1 else [bass_pitch]
```

```
# 2. Iterate through the discrete rhythmic hits defined in the pattern
for hit in pattern.hits:

    # Calculate the base beat position relative to the global track timeline
    hit_global_time = start_measure_time + hit.beat_offset

    # 3. Apply Groove/Swing Micro-timing to the global timeline float
    swung_time = apply_swing(
        time_in_beats=hit_global_time,
        swing_percent=swing_percent,
        swing_grid=0.25  # Standard 16th note swing for Neo-Soul
    )

    # Determine exactly which subset of pitches is triggered by this specific hit
    target_pitches =
    if hit.voice_group == "bass":
        target_pitches = [bass_pitch]
    elif hit.voice_group == "chord":
        target_pitches = upper_voices
    else:
        target_pitches = chord_pitches

    # 4. Temporal Displacement (Chord Rolling)
    # Calculates staggered start times for however many notes exist in this hit array
    rolled_times = calculate_roll_offsets(
        pitches=target_pitches,
        base_time=swung_time,
        bpm=bpm,
        roll_ms_per_note=15.0, # Target a 15ms roll gap for a lush R&B feel
        randomize_roll=True
    )

    total_voices = len(target_pitches)

    # 5. Iteratively write individual humanized notes to midiutil
    for i, pitch in enumerate(target_pitches):

        # Apply duration humanization (subtle truncation/extension for legato realism)
        # Varying note cutoff by a Gaussian distribution of +/- 4%
        dur_jitter = random.gauss(mu=1.0, sigma=0.04)
        final_duration = hit.duration * max(0.8, min(1.2, dur_jitter))
```

```
        # Apply velocity humanization through the comprehensive metrical engine
        final_velocity = humanize_velocity(
            beat_offset=hit.beat_offset,
            pattern_multiplier=hit.velocity_mult,
            voice_index=i,
            total_voices=total_voices,
            base_velocity=base_velocity,
            humanize_amount=6.0  # Allow a standard deviation of 6 velocity units
        )

        # Extract the precise, rolled float time from the calculated array
        final_time = rolled_times[i]

        # 6. Commit the highly processed event to the midiutil architecture
        midi_object.addNote(
            track=track,
            channel=channel,
            pitch=pitch,
            time=final_time,
            duration=final_duration,
            volume=final_velocity  # Historically, midiutil maps velocity to the 'volume' param
        )
```

Expanding the Architecture for Generative Evolution

The architecture outlined above successfully translates a deterministic array of hits into a humanized measure. However, repeating a single measure pattern endlessly throughout a track, even with micro-timing humanization, eventually induces listener fatigue.[42] To elevate the system to true algorithmic composition, a Markov-chain or probabilistic state-machine can be layered atop the CompingPattern selection logic.[22]

By defining a broad repository of compatible patterns—for instance, one pattern featuring straight block comping, another featuring a heavy Neo-soul push, and a third featuring rapid, syncopated ghost notes—the Python engine can randomly select or algorithmically transition between patterns every measure.[22] This ensures that while the intra-measure micro-timing (the J Dilla swing math and Gaussian velocity distribution) precisely mimics human biomechanics, the inter-measure rhythm selection mirrors the spontaneous, improvisational creativity of a live instrumentalist.[42] Furthermore, adjusting global parameters programmatically—such as mapping a dynamic crescendo by steadily increasing the base_velocity over an eight-measure bridge—transforms the output from simple looping backing tracks into highly expressive, evolving musical compositions.[6]

Conclusion

Translating rigid, symbolically parsed chord data into fluid, stylistically complex MIDI performances within a pure Python environment demands a rigorously defined mathematical and object-oriented architecture. By sidestepping bloated external frameworks and relying entirely on Python's foundational data structures alongside the primitive midiutil output capabilities, an exceptionally scalable, mathematically sound, and precise comping engine can be engineered.

This architecture systematically satisfies complex humanization constraints through an interplay of explicit logic and stochastic distribution. Rhythm patterns are effectively abstracted into decoupled schemas, establishing an independent metrical grid that respects the stylistic boundaries of Neo-soul and R&B.[1] MPC-style swing is resolved computationally by dynamically calculating metrical index positions and delaying odd-indexed grid float fractions, simulating the nuanced, lopsided shuffle of legendary producers.[11] Velocity generation is removed from the realm of chaotic, uniform randomness and grounded instead in metrical weight, filtered through the biomechanical realities of outer-voice finger emphasis, and randomized using Gaussian normal distributions to simulate human intent versus error.[12] Finally, the physiological limitations of a pianist's hands are faithfully emulated by mathematically calculating millisecond offset variables against the active BPM track tempo, producing lush, staggered chord rolls.[37]

When aggregated within the master synthesis loop, this framework successfully overwrites the robotic default of standard software sequence generation. It proves conclusively that expressive, authentic algorithmic music is not solely reliant on opaque machine-learning models or massive digital audio workstations. Rather, true musical groove can be deterministic, deeply mathematical, and elegantly encapsulated in clean, structured Python code.

Works cited

1. Pytakt: a Python library for symbolic music description, generation, and real-time processing, accessed March 25, 2026, https://www.tandfonline.com/doi/full/10.1080/09298215.2025.2540434
2. Algorithmic composition - Wikipedia, accessed March 25, 2026, https://en.wikipedia.org/wiki/Algorithmic_composition
3. 4 Steps to Play Neo Soul Chords on Piano, accessed March 25, 2026, https://pianowithjonny.com/piano-lessons/4-steps-to-play-neo-soul-chords-on-piano/
4. Algorithmic Composition - Chan Jun Shern, accessed March 25, 2026, https://junshern.github.io/algorithmic-music-tutorial/part1.html
5. Overview of MIDI Transform presets in Logic Pro for Mac - Apple Support (GU), accessed March 25, 2026, https://support.apple.com/en-gu/guide/logicpro/lgcp215831be/mac
6. MIDIUtil crescendo: Python - Stack Overflow, accessed March 25, 2026, https://stackoverflow.com/questions/42834826/midiutil-crescendo-python

7. Neo-Soul Rhythm : r/musictheory - Reddit, accessed March 25, 2026, https://www.reddit.com/r/musictheory/comments/tjjcpg/neosoul_rhythm/
8. duration of isolated 16th notes in the score (conversion from midi using musescore) · Issue #20 · MarkCWirt/MIDIUtil - GitHub, accessed March 25, 2026, https://github.com/MarkCWirt/MIDIUtil/issues/20
9. How many velocity 'layers' do the beginner to mid range digital pianos have? - Reddit, accessed March 25, 2026, https://www.reddit.com/r/piano/comments/6zbr6s/how_many_velocity_layers_do_the_beginner_to_mid/
10. BobTheScientist/humanizer2 - GitHub, accessed March 25, 2026, https://github.com/BobTheScientist/humanizer2
11. The Dilla Feel, Part III: The Grooves (Real-World Examples and Dilla's Influence), accessed March 25, 2026, https://www.brltheory.com/analysis/dilla-part-iii-grooves-real-world-examples-dillas-influence/
12. Python Generate a random Maxwell distribution from a normal distribution - Stack Overflow, accessed March 25, 2026, https://stackoverflow.com/questions/61717317/python-generate-a-random-maxwell-distribution-from-a-normal-distribution
13. 3 Neo Soul Compositions FULLY EXPLAINED - YouTube, accessed March 25, 2026, https://www.youtube.com/watch?v=SbFQQMEN700
14. The Perfect Neo Soul Progression For Beginners - YouTube, accessed March 25, 2026, https://www.youtube.com/watch?v=RPSop84iY1c
15. Comping Rhythms - 10 Examples You Need To Know - Jens Larsen, accessed March 25, 2026, https://jenslarsen.nl/WP/wp-content/uploads/2019/12/Comping-Rhythms-10-Examples-You-Need-To-Know-PDF.pdf
16. How to Humanize MIDI Like a True Professional (20+ Pro Tips) - Unison Audio, accessed March 25, 2026, https://unison.audio/how-to-humanize-midi/
17. How to humanize your drums: 3 ways to breathe life into your MIDI - Blog | Splice, accessed March 25, 2026, https://splice.com/blog/humanize-your-drums/
18. Introduction - Musicpy - Read the Docs, accessed March 25, 2026, https://musicpy.readthedocs.io/en/latest/Introduction/
19. Best way to humanize quantized MIDI - Cubase - Steinberg Forums, accessed March 25, 2026, https://forums.steinberg.net/t/best-way-to-humanize-quantized-midi/144229
20. Where the accent should be when the beat is made up of different notes - Music, accessed March 25, 2026, https://music.stackexchange.com/questions/103591/where-the-accent-should-be-when-the-beat-is-made-up-of-different-notes
21. Algorithmic Composition: The Music of Mathematics, accessed March 25, 2026, https://www.hsc.edu/documents/academics/MathCS/Pendergrass/algorithmicCompositionPaper_HSCHonors.pdf
22. Week 5 – Music Generation and Algorithmic Composition - CMU School of Computer Science, accessed March 25, 2026,

https://www.cs.cmu.edu/~music/cmp/archives/cmsip/slides/05-algo-comp.pdf
23. 5 COMPING Piano Rhythm Patterns for ANY SITUATION - YouTube, accessed March 25, 2026, https://www.youtube.com/watch?v=5ROx6wAMRg8
24. R&B & Neosoul Free MIDI Pack - MusicCreator, accessed March 25, 2026, https://musiccreator.com/products/r-b-neosoul-free-midi-pack
25. 15 R&B Chord Progressions That SOUND FIRE (Free MIDI Pack) - YouTube, accessed March 25, 2026, https://www.youtube.com/watch?v=h2VkZKoWo34
26. Data structures of musicpy - GitHub, accessed March 25, 2026, https://github.com/Rainbow-Dreamer/musicpy/wiki/Data-structures-of-musicpy
27. Editing MIDI Notes and Velocities — Ableton Reference Manual Version 11, accessed March 25, 2026, https://www.ableton.com/en/live-manual/11/editing-midi-notes-and-velocities/
28. TruSwing - Swing Tempo, accessed March 25, 2026, https://www8.garmin.com/manuals/webhelp/truswing/EN-US/GUID-CC3A61DE-E455-4192-9DA6-C8B0B4EC4A07.html
29. Microtiming: Degrees of Swing, Reverse Swing, and Compound ..., accessed March 25, 2026, https://jackashley.co.uk/2020/08/12/microtiming/
30. Swing setting - Feature Requests - Singular Sound Forum, accessed March 25, 2026, https://forum.singularsound.com/t/swing-setting/4535
31. Swing / Shuffle Percentages explained using MIDI on Pro Tools. - YouTube, accessed March 25, 2026, https://www.youtube.com/watch?v=AYo5XPoBxN0
32. Understanding How SWING RATIOS Works - YouTube, accessed March 25, 2026, https://www.youtube.com/watch?v=r5kBJ15H7Wk
33. The math of drunk swing (Dilla swing, Septuplet swing) - YouTube, accessed March 25, 2026, https://www.youtube.com/watch?v=LQbj48F2Pwg
34. Understanding Your Swing Metrics: Swing Tempo Explained - My Framer Site, accessed March 25, 2026, https://www.swingtuneup.com/support/swing-metrics-swing-tempo
35. Can someone explain to me why swing affects 16th notes? - Elektronauts, accessed March 25, 2026, https://www.elektronauts.com/t/can-someone-explain-to-me-why-swing-affects-16th-notes/70160
36. guide on dynamics ranges vs. midi velocity values? - Logic Pro Help, accessed March 25, 2026, https://www.logicprohelp.com/forums/topic/22984-guide-on-dynamics-ranges-vs-midi-velocity-values/
37. Calculate shift in milliseconds based on BPM - Logic Pro Help, accessed March 25, 2026, https://www.logicprohelp.com/forums/topic/145670-calculate-shift-in-milliseconds-based-on-bpm/
38. How to convert ms (milliseconds) to BPM (beats per minute) and vice versa, accessed March 25, 2026, https://guitargearfinder.com/guides/convert-ms-milliseconds-bpm-beats-per-minute-vice-versa/
39. Tempo BPM to MS Calculator | Tuneform, accessed March 25, 2026,

https://tuneform.com/tools/time-tempo-bpm-to-milliseconds-ms
40. Time interval (in ms) from BPM (Midi tempo) - Stack Overflow, accessed March 25, 2026, https://stackoverflow.com/questions/9675031/time-interval-in-ms-from-bpm-midi-tempo
41. Piano roll Controls - FL Studio, accessed March 25, 2026, https://www.image-line.com/fl-studio-learning-content/fl-studio-online-manual/html/pianoroll.htm
42. A new algorithmic MIDI Sequencer in pure Python (Open Source) - Reddit, accessed March 25, 2026, https://www.reddit.com/r/algorithmicmusic/comments/1rht3oa/a_new_algorithmic_midi_sequencer_in_pure_python/